Reactive SDN for Securing ICS Environments

A Literature Review

Research project: an SDN-based system for dynamic security in industrial control networks · July 2026

Abstract

Industrial control system (ICS) networks were built for availability and determinism, not for the adversarial internet-facing conditions they now operate under. Many field protocols carry no authentication or integrity guarantees, and the strict real-time and safety requirements of the physical process make conventional, intrusive security controls difficult to deploy. Software-defined networking (SDN) has emerged as a promising remedy because it decouples the control plane from the data plane, giving a logically centralized controller a global view of the network and the ability to reprogram forwarding behaviour on demand. This review surveys how SDN has been used to provide *reactive* security in ICS: systems that consume event indicators from external services such as an intrusion detection system (IDS) and modify network routing in response — blocking unseen devices, mirroring traffic for deeper inspection, redirecting critical flows to honeypots, or re-routing around compromised nodes to preserve the process. It organizes the field using the intrusion-response taxonomy of Etxezarreta et al. (2023), examines the IDS-to-controller integration pattern that sits at the core of the proposed project, reviews the Ryu / Mininet / MiniCPS tooling commonly used for evaluation, and identifies the open gaps this project is positioned to address.

1. Introduction and Scope

Attacks against ICS and SCADA infrastructure have repeatedly demonstrated that these environments are no longer isolated. The convergence of operational technology (OT) with corporate IT networks, remote maintenance access, and the use of commodity hardware and TCP/IP-based field protocols has broadened the attack surface considerably. Yet the defensive options available in ICS are constrained: field devices are long-lived and rarely patched, protocols such as Modbus/TCP and DNP3 were designed without security in mind, and the overriding requirement is that the physical process must keep running safely. A security control that drops legitimate control traffic, adds unpredictable latency, or takes the controller offline can itself become a safety incident.

SDN offers a way out of this bind. Because forwarding decisions are made by a programmable controller rather than by fixed switch firmware, defenders gain a single point from which to observe the whole network and to install, update, or revoke flow rules in near-real time. This makes it feasible to build *dynamic* defenses that react to events rather than relying solely on static perimeter rules. The specific problem addressed by this project is the construction of such a reactive system: one that receives event indicators from external services (most importantly an IDS) and modifies network routing in response, choosing an action that is appropriate to the device and the flow without compromising the safety of the process.

This review is scoped accordingly. It concentrates on SDN-based intrusion response and dynamic security for ICS/SCADA, on the integration of external detection with the SDN control plane, and on the tools and testbeds used to evaluate such systems. It does not attempt to survey ICS intrusion detection algorithms in depth, treating the IDS as an external event source, which mirrors the architecture of the proposed system.

2. The ICS Security Landscape and the Case for SDN

The security requirements of ICS differ markedly from enterprise IT. The classic confidentiality-integrity-availability priority is effectively inverted: availability and integrity of control dominate, and any measure that threatens deterministic, real-time communication is viewed with suspicion. NIST SP 800-82 codifies these constraints and repeatedly stresses that security controls must not interfere with the safe and reliable operation of the process. Legacy protocols compound the problem, since many carry no authentication, allowing an attacker who reaches the network to spoof commands or inject false sensor readings.

Against this backdrop, SDN's appeal is its **programmability and global visibility**. A logically centralized controller can enforce fine-grained, per-flow policy; it can distinguish an “unseen” device from a known one because it holds network-wide state; and it can reconfigure paths without physically touching switches. Surveys of the area note that this centralized view enables coordinated, automated responses to network events and classified incidents, providing timely and accurate incident-response actions that are simply not available with static, distributed configuration. The same programmability underpins the OpenFlow abstraction and the emerging use of P4 for programmable data planes, both of which appear repeatedly in ICS security proposals.

The centralization is double-edged, however, and the literature is candid about it. The controller becomes a high-value target and a potential single point of failure: compromising or overwhelming it can bring down the whole network, and the control channel itself is an attractive target for denial-of-service. Rule installation latency and controller throughput can become bottlenecks precisely when the network is under load. These concerns temper the enthusiasm and motivate design choices — controller resilience, lightweight data-plane filtering, and careful attention to real-time behaviour — that recur throughout the reactive-defense work reviewed below.

3. A Taxonomy of SDN-Based Intrusion Response in ICS

The most complete organizing framework for this space is the survey by Etxezarreta, Garitano, Iturbe and Zurutuza (2023), which classifies SDN-based intrusion-response strategies for ICS into four families. The table below summarizes them; the subsections that follow discuss each in turn. A cross-cutting observation from the survey is that most proposals implement their response logic in the control plane, with only a minority pushing enforcement down to the data plane (for example via P4), a distinction that matters directly for real-time performance.

Strategy class	Core idea	Representative works
Dynamic traffic filtering	Install / update flow rules to allow, drop, mirror or redirect flows in reaction to alerts; allowlists for ICS protocols.	Piedrahita et al.; Ndonda et al. (P4/Modbus); Tsuchiya et al.; Brugman et al.; Rivera et al.

Strategy class	Core idea	Representative works
Network survivability & reconfiguration	Re-route around compromised nodes / links to keep the process running under attack.	Chavez et al.; Salazar et al.; FDI-mitigation via reconfiguration
Moving Target Defense (MTD)	Proactively/reactively randomize IP addresses and flow paths to shrink the attacker's reconnaissance window.	Shi et al. (CHAOS); Aydeger et al.; MTD routing for smart grid
Honeypot-based response	Redirect suspect traffic to a decoy ICS process for containment and deep inspection.	Bernieri et al. (MimePot); Petroulakis et al.; Du et al.

3.1 Dynamic Traffic Filtering

This is the family most directly aligned with the project's objective. The controller installs or updates flow rules — allow, drop, mirror, or redirect — in reaction to detected conditions. Piedrahita et al. present a canonical example: an SDN controller works with an IDS deployed on an NFV infrastructure to mitigate attacks launched from a compromised water-level sensor. The IDS receives a copy of the sensor measurements and compares them against a model-based estimate; when the deviation exceeds a threshold it notifies the controller, which substitutes anomalous readings with estimated values so that the physical process keeps running safely. This work is a clear demonstration of reacting to an incident *without* compromising process safety, and it was evaluated by extending the MiniCPS emulation environment.

Ndonda and Sadre push enforcement into the data plane with a two-level IDPS: the first level is a P4-based allowlist filter for Modbus running on the switches themselves, and any packet that fails to match is escalated to a second-level deep-packet-inspection engine (Bro/Zeek) on a dedicated host. Crucially, when the second level detects an intrusion it updates the first-level allowlist, so the same attack is subsequently caught at line rate. This closes the detection-to-response loop while respecting real-time constraints — a design pattern of direct relevance to the project. Tsuchiya et al. build an SDN-based ICS firewall combining a transparent data-plane firewall, temporal filtering that keeps switch rules current, and spatial filtering driven by OPC UA authorization. Brugman et al. place a signature-based IDPS and a deep-packet inspector in the cloud via NFV, with the controller acting as an IP/protocol firewall and dropping flows that analysis flags. Rivera et al. add a policy-engine abstraction on the controller that translates domain-specific rules (allow, drop, log, copy) into SDN actions, and Melis et al. focus on making policy definition and verification easier for operators.

3.2 Network Survivability and Reconfiguration

Where filtering blocks or inspects individual flows, survivability approaches keep the process alive by re-routing around compromised or degraded elements. SDN's ability to recompute paths centrally makes this natural. A representative line of work uses network reconfiguration to defeat stealthy false-data-injection (FDI) attacks in smart-grid SCADA by exploiting the attacker's uncertainty about the topology; reported results prevent a large majority of stealthy FDI attempts while maintaining SCADA performance. For a reactive security system, survivability is the safety net: if a device must be isolated, the controller should where possible preserve connectivity for the remaining legitimate control loops rather than partitioning the process.

3.3 Moving Target Defense

Moving Target Defense (MTD) turns SDN's programmability toward proactive obfuscation, continually changing network properties so that the attacker's reconnaissance decays in value. Typical mechanisms

randomize IP addresses and flow paths, or introduce decoy servers. Shi et al.'s CHAOS is a widely cited SDN-based MTD system that obfuscates hosts, ports and paths, and later work applies MTD routing specifically to SDN-enabled smart grids. The survey notes that MTD can be applied either proactively or reactively — randomizing addresses/paths on a schedule, or triggering re-randomization in response to an alert — which makes it complementary to the reactive filtering at the heart of this project. A recognized caveat is that address/path churn must be reconciled with ICS devices that expect stable, deterministic communication.

3.4 Honeypot-Based Response

The fourth family redirects suspect traffic to a decoy process, achieving containment and gathering intelligence at the same time. SDN is a good fit because the controller can transparently steer a flagged flow to a honeypot without the attacker's cooperation. Bernieri et al.'s MimePot models the physical process and, on detecting a data-integrity attack, has the controller redirect malicious traffic to the decoy. Petroulakis et al. use a service-chaining function to forward malicious traffic to a honeypot, and other work embeds an SDN controller inside a water-treatment honeypot to manage it dynamically. For the project, this family supplies the “redirect for deeper inspection” response option cited in the problem statement — appropriate when a flow is suspicious but blocking outright would risk a critical service.

4. Reactive IDS-to-Controller Integration

The mechanism common to the works above — and the specific focus of this project — is the pipeline from external detection to network action. In the general SDN literature this appears as anomaly-based dynamic access control: an IDS captures a network anomaly and generates or triggers SDN flow rules to enforce access control, with the controller producing a new rule according to the detection result so that offending traffic is filtered. The vocabulary of possible responses is well established and maps cleanly onto the project's requirements:

- **Allow** — explicitly permit a known-good flow to proceed.
- **Block / quarantine** — deny a flow outright, the right action for an unseen device that triggers an alert.
- **Mirror** — permit the flow but send a copy to a monitoring or DPI service for deeper inspection.
- **Redirect** — steer the flow to another device, such as a honeypot or forensic appliance, without interrupting the rest of the network.

The two installation strategies for these rules — *reactive* (rules installed on demand when the first packet of a flow reaches the controller) and *proactive* (rules pre-installed) — present a trade-off that the ACL-change literature analyses directly: reactive installation gives maximum flexibility and dynamic control of forwarding but adds first-packet latency and increases controller load, whereas proactive installation is faster at forwarding time but less adaptive. A practical reactive ICS system typically blends the two, pre-installing rules for known critical control loops while reacting to alerts for anomalous or unseen traffic. A related design lesson from the general SDN security literature is that inspection appliances rarely have the bandwidth to examine all traffic, so mirroring/redirection is usually applied selectively to a subset of flows rather than universally.

An important architectural theme, emphasized by Piedrahita et al. and by the survey, is **safety-preserving response**. In an ICS the controller cannot treat “block everything suspicious” as a safe default, because cutting a control loop can endanger the process. The more sophisticated proposals therefore couple the network

action with process awareness — substituting estimated values for anomalous sensor readings, isolating a device while re-routing legitimate traffic, or redirecting rather than dropping a critical service. This is precisely the balance the project's objective calls for.

5. Tools, Testbeds, and Evaluation Environments

Because production ICS cannot be experimented on, the field relies heavily on emulation, and a fairly standard toolchain has emerged — one well matched to the project's choice of the **Ryu** controller. Ryu is a lightweight, Python-based SDN framework in which defense logic is written as controller applications, which makes it a common choice for research prototypes where reactive flow logic must be scripted quickly. It is typically paired with **Mininet** and Open vSwitch to emulate the network: reported testbeds run Ryu on an Ubuntu VM with Mininet establishing the topology, and IDS functionality implemented as a native Ryu application. Such studies also quantify the overhead of adding detection — in one Ryu/Mininet testbed the IDS raised average latency by only ~0.016 ms and CPU usage by ~5%, evidence that the approach is compatible with ICS timing budgets when engineered carefully.

For the *industrial* side of the testbed, MiniCPS (Antonioli and Tippenhauer) is the de-facto standard for security research on cyber-physical networks and underpins several of the reactive-response works discussed above, including Piedrahita et al.'s incident-response mechanism. Field-protocol realism is usually provided by Modbus/TCP — for example a two-tank process with PLCs as Modbus slaves and an HMI as the Modbus master — with DNP3 also appearing. Programmable data-plane experiments use P4 to implement allowlist filtering directly on switches. In short, a Ryu + Mininet/Open vSwitch + MiniCPS + Modbus stack is both well-trodden and directly suited to building and evaluating the proposed reactive system, with P4 as an option should data-plane enforcement be desired for the most latency-sensitive filtering.

6. Safety and Real-Time Constraints

A thread running through the whole literature is that reactivity must not come at the expense of the process. Three constraints recur. First, timing: control loops have deadlines, so any response that adds jitter or delay — first-packet rule installation, redirection through a DPI appliance — must stay within budget, motivating data-plane filtering and selective inspection. Second, availability of the control plane itself: since the controller is a single point of failure and a DoS target, resilient or multi-controller designs and lightweight localized mitigation at the switch level are recurring recommendations. Third, correctness of the response: an over-aggressive automated action can cause the very outage it was meant to prevent, which is why the strongest proposals make responses process-aware and reversible. These constraints are not peripheral; they are the design envelope within which any reactive ICS-SDN system, including this project, must operate.

7. Research Gaps and Positioning of This Project

Several gaps emerge from the reviewed work and define where this project can contribute:

- **Generic, source-agnostic event ingestion.** Most systems bind tightly to one detector. A reactive controller that accepts standardized event indicators from arbitrary external services (IDS, asset-discovery, anomaly engines) and maps them to responses would be more broadly deployable — and is exactly the interface the project's objective describes.

- **Device-aware, differentiated responses.** The problem statement's own example — block an unseen device but redirect a critical service for inspection — requires the controller to reason about device identity and criticality. Few systems tie the response choice to a model of which devices and flows are safety-critical.
- **Safety-constrained automation.** While Piedrahita et al. show process-aware response is possible, general frameworks that guarantee an automated action will not violate a defined safety envelope remain scarce.
- **Lightweight data-plane enforcement.** The survey explicitly notes a lack of lightweight architectures that provide localized mitigation at the switch level while still interfacing with the controller — a gap that P4-based filtering only partially fills.
- **Controller resilience under attack.** Reactive designs increase reliance on the control plane precisely when it may be targeted; combining reactive response with controller-resilience mechanisms is under-explored in the ICS setting.

Positioned against these gaps, the proposed system — an SDN controller (Ryu) that ingests event indicators from external services such as an IDS and modifies routing to block, mirror, redirect, or re-route in a safety-preserving, device-aware manner — sits squarely within the dynamic-traffic-filtering and honeypot-response families while addressing their most cited shortcomings. Building it on the established Ryu / Mininet / MiniCPS / Modbus toolchain allows direct, reproducible comparison with the prior work reviewed here, and leaves a clear path toward the harder open problems of safety-constrained automation and controller resilience.

References

[1] Etxezarreta, X., Garitano, I., Iturbe, M., & Zurutuza, U. (2023). Software-Defined Networking approaches for intrusion response in Industrial Control Systems: A survey. International Journal of Critical Infrastructure Protection, 42, 100615.

[2] Piedrahita, A. F. M., Gaur, V., Giraldo, J., Cárdenas, Á. A., & Rueda, S. J. (2018). Leveraging Software-Defined Networking for Incident Response in Industrial Control Systems. IEEE Software, 35(1), 44–50.

[3] Ndonda, G. K., & Sadre, R. (2018). A Two-level Intrusion Detection System for Industrial Control System Networks using P4. In Proc. 5th Int. Symposium for ICS & SCADA Cyber Security Research (ICS-CSR).

[4] Shi, Y., Zhang, H., Wang, J., Xiao, F., Huang, J., Zou, D., et al. (2017). CHAOS: An SDN-Based Moving Target Defense System. Security and Communication Networks, 2017, 3659167.

[5] Bernieri, G., Conti, M., & Pascucci, F. (2019). MimePot: a Model-based Honeypot for Industrial Control Networks (SDN-based traffic redirection). In Proc. IEEE Int. Conf. on Systems, Man and Cybernetics (SMC).

[6] Petroulakis, N. E., et al. Service-chaining based honeypot redirection for ICS using SDN. (As surveyed in Etxezarreta et al., 2023.)

[7] Tsuchiya, A., Fraile, F., Koshijima, I., Ortiz, A., & Poler, R. Software-defined firewall for securing industrial control systems (transparent, temporal and spatial/OPC UA filtering). (As surveyed in Etxezarreta et al., 2023.)

[8] Brugman, J., et al. Cloud-based intrusion detection and prevention system for industrial control systems using SDN/NFV. (As surveyed in Etxezarreta et al., 2023.)

[9] Rivera, S., et al. SDN policy-engine security mechanism for robotic/CPS systems (allow, drop, log, copy actions). (As surveyed in Etxezarreta et al., 2023.)

[10] Antonioli, D., & Tippenhauer, N. O. (2015). MiniCPS: A Toolkit for Security Research on CPS Networks. In Proc. 1st ACM Workshop on Cyber-Physical Systems-Security and/or PrivaCy (CPS-SPC), 91–100.

[11] McKeown, N., Anderson, T., Balakrishnan, H., Parulkar, G., Peterson, L., Rexford, J., Shenker, S., & Turner, J. (2008). OpenFlow: Enabling Innovation in Campus Networks. ACM SIGCOMM Computer Communication Review, 38(2), 69–74.

[12] Lantz, B., Heller, B., & McKeown, N. (2010). A Network in a Laptop: Rapid Prototyping for Software-Defined Networks. In Proc. 9th ACM SIGCOMM Workshop on Hot Topics in Networks (HotNets).

[13] Ryu SDN Framework Community. Ryu: a component-based software-defined networking framework (Python). https://ryu-sdn.org

[14] Stouffer, K., Pillitteri, V., Lightman, S., Abrams, M., & Hahn, A. (2015). Guide to Industrial Control Systems (ICS) Security. NIST Special Publication 800-82 Rev. 2.

Note on sources: the classification and several primary works are drawn from the Etxezarreta et al. (2023) survey, which serves as the anchor reference; individual proposals attributed to that survey are listed for traceability and should be cited to their original publications when quoted directly.